Adaptive DC Motor Speed Control with Drivetrain Compliance

Design physics-aware speed control for a DC motor driving a load through a flexible shaft. Keeping the reflected load inertia J_L symbolic exposes how drivetrain compliance shifts the torsional resonance, then produces state-feedback gains $K(J_L)$ that adapt across traction and vehicle-load conditions without re-tuning.

Workflow: derive the symbolic 4-state motor-shaft-load model → substitute motor and shaft constants while sweeping J_L → analyze open-loop pole migration and resonance shift → place closed-loop poles symbolically for $K(J_L)$ → generate adaptive gain and reference-scaling functions with `matlabFunction` → validate speed tracking across lost-traction and heavy-load cases

Requires: Symbolic Math Toolbox™, Control System Toolbox™. Optional: Simulink®

Copyright 2026 The MathWorks, Inc.

1. Symbolic System Modeling

Four states model the motor-shaft-load drivetrain: motor speed ω_m, armature current i, shaft twist angle $\Delta\theta$, and load speed ω_L. Input: armature voltage V. Output: load speed ω_L. The flexible shaft coupling (stiffness K_s, damping C_s) between motor and load creates a torsional resonance, which is the key dynamic that shifts with J_L.

Governing equations:

- Motor: $J_m\dot{\omega}_m = K_m i - b_m\omega_m - K_s\Delta\theta - C_s(\omega_m - \omega_L)$
- Electrical: $L\dot{i} = V - Ri - K_m\omega_m$
- Shaft: $\dot{\Delta\theta} = \omega_m - \omega_L$
- Load: $J_L\dot{\omega}_L = K_s\Delta\theta + C_s(\omega_m - \omega_L) - b_L\omega_L$

Physical parameters: J_m - motor inertia [kg·m²], J_L - load inertia, b_m/b_L - friction [N·m·s], K_m - motor constant [V/(rad/s)], R - resistance [Ω], L - inductance [H], K_s - shaft stiffness [N·m/rad], C_s - shaft damping [N·m·s]. Assume all are >0.

```
syms J_L J_m b_m b_L Km R L K_s C_s positive
A = [(-b_m - C_s)/J_m,  Km/J_m,  -K_s/J_m,   C_s/J_m;
     -Km/L,              -R/L,     0,          0;
```

```
      1,                0,        0,          -1;
      C_s/J_L,          0,        K_s/J_L,    (-b_L - C_s)/J_L]
```

A =

$$\begin{bmatrix} -\frac{C_s + b_m}{J_m} & \frac{\mathrm{Km}}{J_m} & -\frac{K_s}{J_m} & \frac{C_s}{J_m} \\ -\frac{\mathrm{Km}}{L} & -\frac{R}{L} & 0 & 0 \\ 1 & 0 & 0 & -1 \\ \frac{C_s}{J_L} & 0 & \frac{K_s}{J_L} & -\frac{C_s + b_L}{J_L} \end{bmatrix}$$

```
B = [0; 1/L; 0; 0]
```

B =

$$\begin{bmatrix} 0 \\ \frac{1}{L} \\ 0 \\ 0 \end{bmatrix}$$

```
C = [0 0 0 1];
D = 0;
```

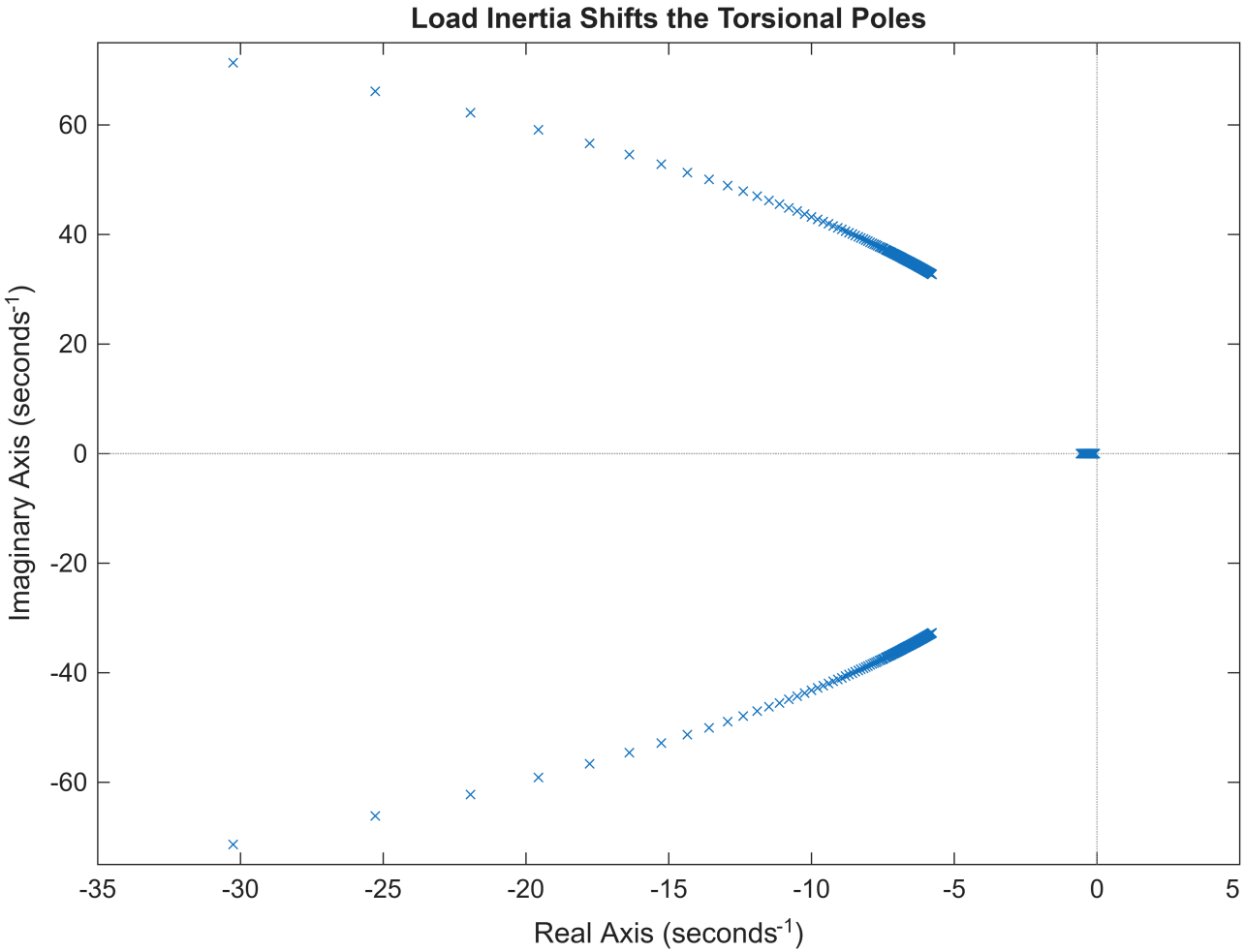
2. Open-Loop Dynamics: How Load Inertia Affects the Plant

Substitute known motor and shaft constants but keep J_L symbolic. Sweeping J_L reveals two key effects: the plant poles migrate in the complex plane, and the torsional resonance peak shifts in the frequency response. This is the core problem that motivates adaptive control.

```
J_m_val = 0.05; b_m_val = 0.01; b_L_val = 0.005;
Km_val = 0.1; R_val = 0.5; L_val = 0.005;
K_s_val = 50; C_s_val = 0.5;
motor_params = [J_m b_m b_L Km R L K_s C_s];
motor_values = [J_m_val b_m_val b_L_val Km_val R_val L_val K_s_val C_s_val];
A_motor = subs(A, motor_params, motor_values);
B_motor = subs(B, motor_params, motor_values);
```

```
B_Val = double(B_motor);
C_Val = double(C);
D_Val = double(D);
% Sweep J_L and build an open-loop state-space model array
J_L_range = linspace(0.01, 0.5, 200);
n_pts = length(J_L_range);
open_loop_models = cell(1, n_pts);
for idx = 1:n_pts
    A_n = double(subs(A_motor, J_L, J_L_range(idx)));
    open_loop_models{idx} = ss(A_n, B_Val, C_Val, D_Val);
end
sys_ol_array = stack(1, open_loop_models{:});
sys_ol_array.SamplingGrid = struct('J_L', J_L_range(:));

pole_figure = figure(Name='Open-Loop Pole Migration', ...
    Position=[100 100 760 520]);
pole_plot = pzplot(pole_figure, sys_ol_array, MarkerSize=5);
pole_plot.XLimits = {[-35 5]};
pole_plot.YLimits = {[-75 75]};
title(pole_plot, 'Load Inertia Shifts the Torsional Poles')
```



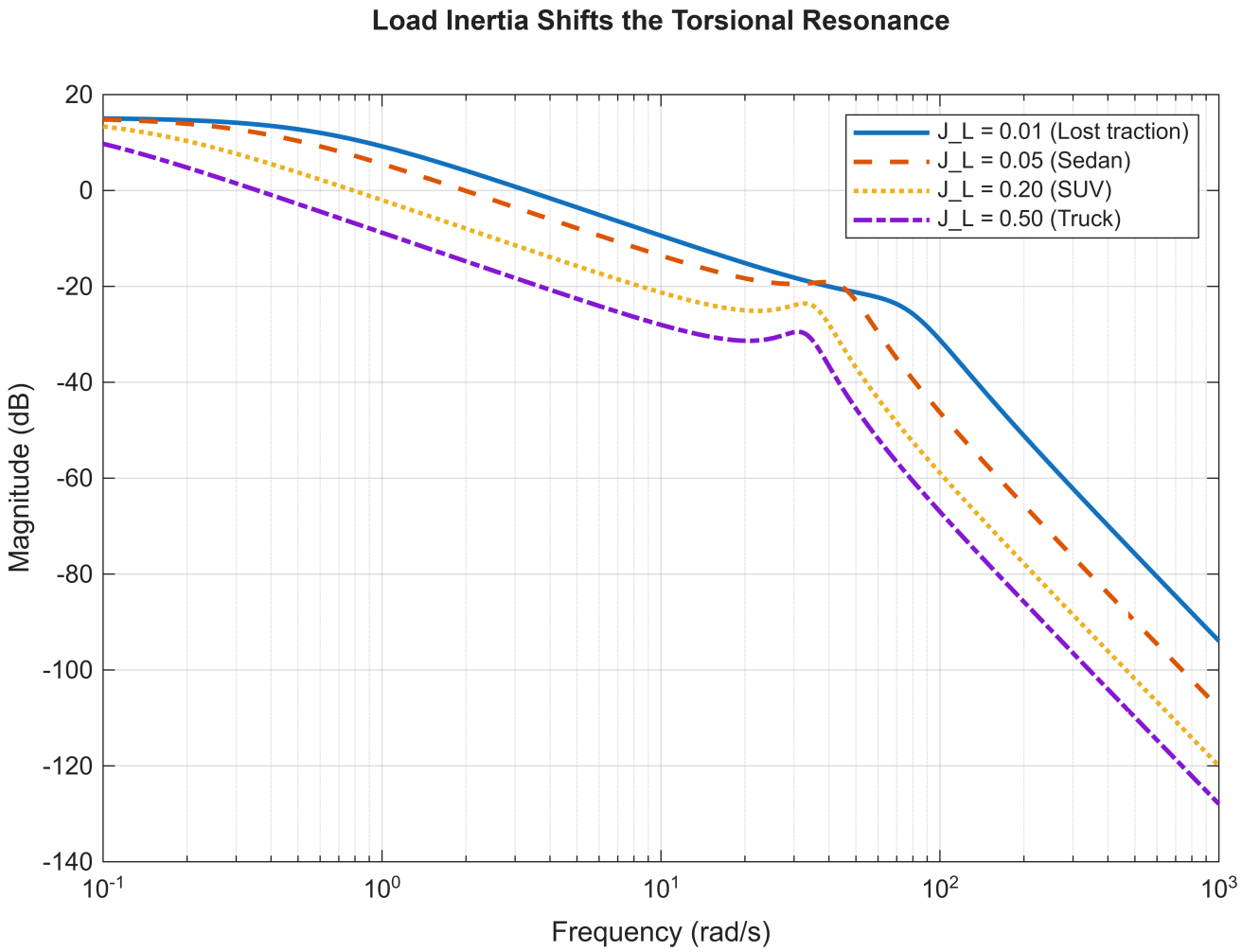
This view focuses on the inertia-sensitive mechanical poles. The fast electrical pole remains near -100 rad/s, outside the displayed range.

```
J_L_bode = [0.01, 0.05, 0.2, 0.5];
labels_bode = ["Lost traction", "Sedan", "SUV", "Truck"];
bode_models = cell(1, length(J_L_bode));
for idx = 1:length(J_L_bode)
    A_n = double(subs(A_motor, J_L, J_L_bode(idx)));
    bode_models{idx} = ss(A_n, B_Val, C_Val, D_Val);
```

```
end
sys_ol_bode = stack(1, bode_models{:});
sys_ol_bode.SamplingGrid = struct('J_L', J_L_bode(:));

% Plot array members as named responses for readable static exports
bode_responses = cell(1, length(J_L_bode));
for idx = 1:length(J_L_bode)
    bode_responses{idx} = sys_ol_bode(:, :, idx);
    bode_responses{idx}.Name = sprintf('J_L = %.2f (%s)', ...
        J_L_bode(idx), labels_bode(idx));
end

w = logspace(-1, 3, 500);
bode_figure = figure(Name='Open-Loop Frequency Response', ...
    Position=[140 140 760 520]);
magnitude_plot = bodemag(bode_figure, ...
    bode_responses{1}, '-', bode_responses{2}, '--', ...
    bode_responses{3}, ':', bode_responses{4}, '-.', ...
    w, LineWidth=1.8);
title(magnitude_plot, 'Load Inertia Shifts the Torsional Resonance')
grid(magnitude_plot, 'on')
legend(magnitude_plot, Location='northeast')
```



Both plots use the parameterized LTI models. Data tips report the J_L value retained from each model's `SamplingGrid`.

3. State Feedback Gains as Functions of J_L

Design a state feedback controller: $u = -Kx + \bar{N}r$. The desired closed-loop poles define the performance spec. Because the system matrix A contains symbolic J_L, the four gains $K = [k_1, k_2, k_3, k_4]$ come out as functions of J_L.

Deployment note: This example assumes full-state feedback. In production hardware, shaft twist angle $\Delta\theta$ is usually estimated rather than measured, so an observer such as a Luenberger observer or EKF would provide $\hat{x}$ from motor current, motor speed, and wheel/load speed sensor data.

```
n = size(A, 1);
Co = [B, A*B, A^2*B, A^3*B];
det_Co = simplify(det(Co))
```

```
det_Co =
```

$$\frac{\mathrm{Km}^3\,(C_s\,b_L - J_L\,K_s)}{J_L{}^2\,J_m{}^3\,L^4}$$

The symbolic determinant identifies exact parameter combinations that cause loss of controllability. Its magnitude is not a reliable numerical test because it depends strongly on units and scaling, so verify the rank after substituting representative parameter values.

```
J_L_check = [0.01, 0.05, 0.2, 0.5];
fprintf('Numerical controllability checks:\n')
```

```
Numerical controllability checks:
```

```
for idx = 1:length(J_L_check)
    A_n = double(subs(A_motor, J_L, J_L_check(idx)));
    controllability_rank = rank(ctrb(A_n, B_Val));
    fprintf('  J_L = %.2f: rank(ctrb(A,B)) = %d of %d\n', ...
        J_L_check(idx), controllability_rank, n);
end
```

```
  J_L = 0.01: rank(ctrb(A,B)) = 4 of 4
  J_L = 0.05: rank(ctrb(A,B)) = 4 of 4
  J_L = 0.20: rank(ctrb(A,B)) = 4 of 4
  J_L = 0.50: rank(ctrb(A,B)) = 4 of 4
```

```
% Desired closed-loop poles:
%   -100       fast electrical mode (near natural L/R pole)
%   -15        moderate real pole
```

```
%   -8 +/- 8i  dominant complex pair (zeta=0.71, settling ~0.5s)
p_des = [-100, -15, -8+8i, -8-8i];

% Symbolic pole placement: solve for K such that poles(A - B*K) = p_des
syms lambda
k = sym('k', [1, n]);
char_eqn = det(A - B*k - lambda*eye(n));
eqns = subs(char_eqn, lambda, p_des);
sol = solve(eqns, k);
K_sym = [sol.k1, sol.k2, sol.k3, sol.k4];
```

State feedback gains K(J_L):

```
% Selective substitution: fix motor/shaft constants, keep J_L free
K_of_JL = simplify(subs(K_sym, motor_params, motor_values))
```

```
K_of_JL =
```

$$\left[\frac{47833600\,J_L^2 - 4240160\,J_L + 10201}{16000000\,J_L^2} \quad \frac{4160\,J_L - 101}{40000\,J_L} \quad -\frac{33600000\,J_L^2 + 2224200\,J_L - 10201}{160000\,J_L^2} \quad \frac{1536000}{}\right.$$

Precompensator $\bar{N}(J_L)$ for Zero Steady-State Error

Compute reference scaling so load speed tracks the setpoint. Since K and the system matrices depend on J_L, so does $\bar{N}$.

```
N_ss = [A, B; C, D] \ [zeros(n, 1); 1];
Nx = N_ss(1:n);
Nu = N_ss(end);
Nbar_sym = Nu + K_sym * Nx;
```

Reference scaling Nbar(J_L):

```
Nbar_of_JL = simplify(subs(Nbar_sym, motor_params, motor_values))
```

```
Nbar_of_JL =
```

$$\frac{48\,J_L}{5}$$

Complete control law: $u = -K(J_L) \cdot x + \bar{N}(J_L) \cdot r$

4. Verification: Desired Poles Are Achieved for Any J_L

Form the closed-loop system and confirm its poles match the desired locations regardless of load inertia. In the unconstrained linear model, step responses show identical settling behavior across the full J_L range.

```
A_cl = A - B*K_sym;
A_cl_of_JL = simplify(subs(A_cl, motor_params, motor_values));
J_L_test = [0.01, 0.05, 0.2, 0.5];
labels = ["Lost traction", "Sedan", "SUV", "Truck"];
closed_loop_models = cell(1, length(J_L_test));
for idx = 1:length(J_L_test)
    J_val = J_L_test(idx);
    A_cl_n = double(subs(A_cl_of_JL, J_L, J_val));
    Nbar_n = double(subs(Nbar_of_JL, J_L, J_val));
    closed_loop_models{idx} = ss(A_cl_n, B_Val*Nbar_n, C_Val, D_Val);
end
sys_cl_array = stack(1, closed_loop_models{:});
sys_cl_array.SamplingGrid = struct('J_L', J_L_test(:));

fprintf('Closed-loop pole verification:\n')
```

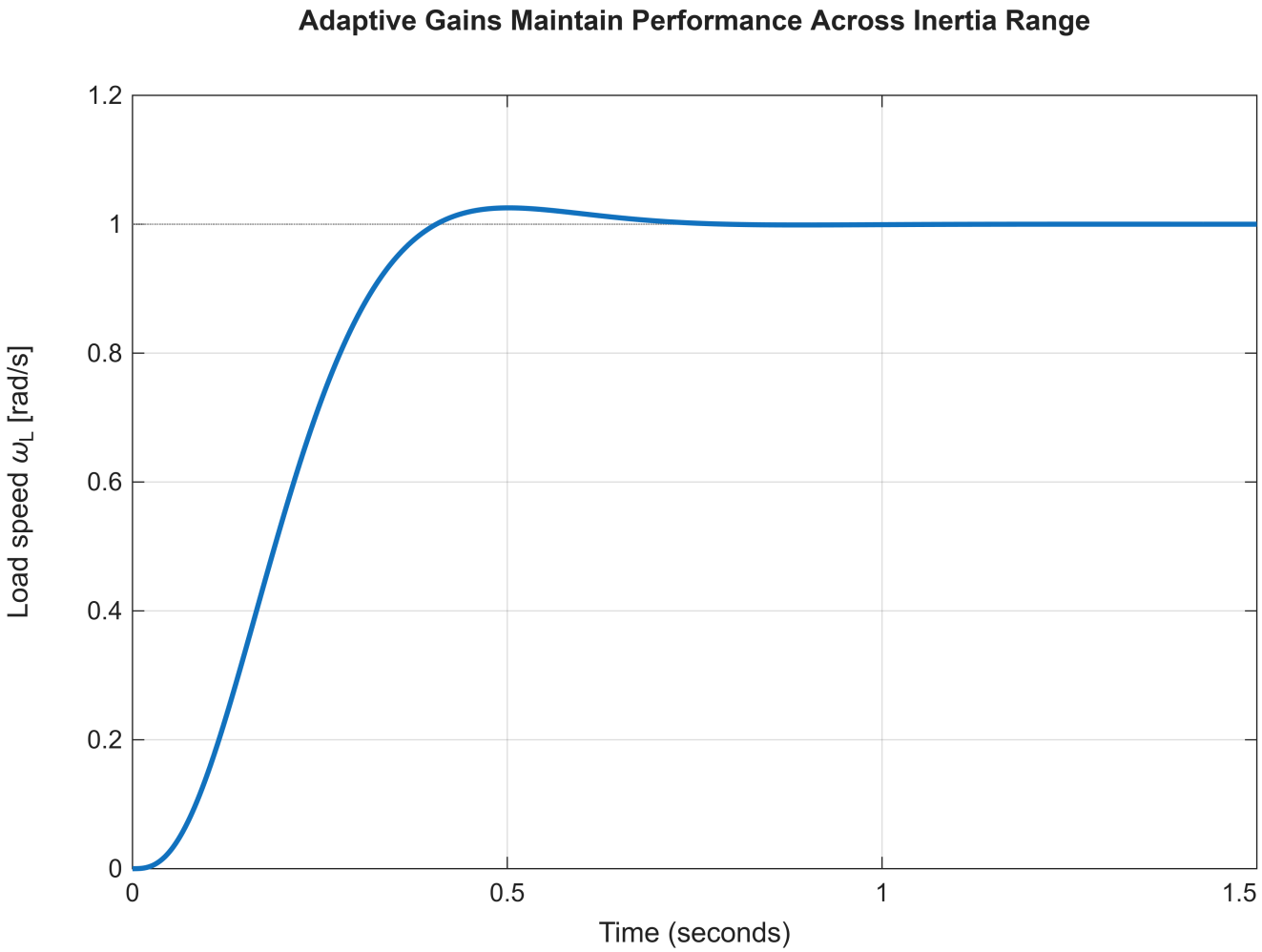
```
Closed-loop pole verification:
```

```
for idx = 1:length(J_L_test)
    poles_cl = esort(pole(sys_cl_array(:, :, idx)));
    fprintf('  J_L = %.2f (%s):', J_L_test(idx), labels(idx));
    for p = 1:length(poles_cl)
        fprintf('  %.1f%+.1fi', real(poles_cl(p)), imag(poles_cl(p)));
    end
    fprintf('\n');
```

```
end
```

```
  J_L = 0.01 (Lost traction):
  -8.0+8.0i  -8.0-8.0i  -15.0+0.0i  -100.0+0.0i
  J_L = 0.05 (Sedan):
  -8.0+8.0i  -8.0-8.0i  -15.0+0.0i  -100.0+0.0i
  J_L = 0.20 (SUV):
  -8.0+8.0i  -8.0-8.0i  -15.0+0.0i  -100.0+0.0i
  J_L = 0.50 (Truck):
  -8.0+8.0i  -8.0-8.0i  -15.0+0.0i  -100.0+0.0i
```

```
% Step responses across the inertia range
clf
t = 0:0.001:1.5;
step_response_plot = stepplot(sys_cl_array, t, LineWidth=2);
title(step_response_plot, 'Adaptive Gains Maintain Performance Across Inertia Range')
ylabel(step_response_plot, 'Load speed \omega_L [rad/s]')
grid(step_response_plot, 'on')
```



Actuator Feasibility: Ideal Poles Meet Hardware Limits

Exact pole placement specifies the unconstrained closed-loop dynamics, but it does not guarantee that the inverter or motor can supply the required voltage and current. Derive exact transfer functions from the speed reference to armature current and control voltage:

$$\frac{I(s)}{R(s)} = e_2^T(sI - A_{cl})^{-1}B\bar{N}, \qquad \frac{V(s)}{R(s)} = \bar{N} - K(sI - A_{cl})^{-1}B\bar{N}$$

```
syms s
state_transfer_of_JL = simplify((s*eye(n) - A_cl_of_JL) \ (B_motor*Nbar_of_JL));
current_transfer_of_JL = simplifyFraction(state_transfer_of_JL(2))
```

current_transfer_of_JL =

$$\frac{48\ \left(50151\,s+1000000\,J_L\,s+10200\,J_L\,s^2+1000\,J_L\,s^3+505\,s^2+15000\right)}{25\ (s+100)\ (s^3+31\,s^2+368\,s+1920)}$$

```
voltage_transfer_of_JL = simplifyFraction(Nbar_of_JL - K_of_JL*state_transfer_of_JL)
```

voltage_transfer_of_JL =

$$\frac{6\ \left(5050300\,s+100000000\,J_L\,s+2060000\,J_L\,s^2+110200\,J_L\,s^3+1000\,J_L\,s^4+100651\,s^2+505\,s^3+3500\right)}{625\ (s+100)\ (s^3+31\,s^2+368\,s+1920)}$$

```
% Verify that both transfer functions retain the assigned closed-loop poles
closed_loop_denominator = collect(simplify(det(s*eye(n) - A_cl_of_JL)), s);
factored_closed_loop_denominator = (s + 100)*(s + 15)*(s^2 + 16*s + 128)
```

factored_closed_loop_denominator = $(s+15)\ (s+100)\ (s^2+16\,s+128)$

```
assert(isAlways(closed_loop_denominator == expand(factored_closed_loop_denominator)))
```

Pole placement fixes the common denominator, while the J_L-dependent numerators determine how much voltage and current each command requires.

```
% Convert the exact polynomial coefficients to numeric evaluators
[current_numerator, current_denominator] = numden(current_transfer_of_JL);
[voltage_numerator, voltage_denominator] = numden(voltage_transfer_of_JL);
current_numerator_fcn = matlabFunction(coeffs(current_numerator, s, 'All'), 'Vars', {J_L});
voltage_numerator_fcn = matlabFunction(coeffs(voltage_numerator, s, 'All'), 'Vars', {J_L});
current_denominator_coeffs = double(coeffs(current_denominator, s, 'All'));
voltage_denominator_coeffs = double(coeffs(voltage_denominator, s, 'All'));
```

Evaluate transient maxima numerically; the extrema of these mixed exponential and oscillatory responses do not have a useful general closed form. For this linear model, peak demands scale directly with the commanded speed change.

```
voltage_limit = 48;  % DC bus voltage [V]
```

```
current_limit = 40;  % Illustrative inverter/motor current limit [A]
J_L_feasibility = unique([J_L_range, J_L_test]);
peak_voltage_per_speed = zeros(size(J_L_feasibility));
peak_current_per_speed = zeros(size(J_L_feasibility));
t_feasibility = 0:0.001:1.5;

for idx = 1:length(J_L_feasibility)
    J_val = J_L_feasibility(idx);
    current_demand_system = tf(current_numerator_fcn(J_val), ...
        current_denominator_coeffs);
    voltage_demand_system = tf(voltage_numerator_fcn(J_val), ...
        voltage_denominator_coeffs);
    current_response = step(current_demand_system, t_feasibility);
    voltage_response = step(voltage_demand_system, t_feasibility);

    peak_voltage_per_speed(idx) = max(abs(voltage_response), [], 'all');
    peak_current_per_speed(idx) = max(abs(current_response), [], 'all');
end

voltage_limited_reference = voltage_limit./peak_voltage_per_speed;
current_limited_reference = current_limit./peak_current_per_speed;
feasible_reference = min(voltage_limited_reference, current_limited_reference);

[~, summary_indices] = ismember(J_L_test, J_L_feasibility);
feasibility_summary = table(J_L_test(:), ...
    peak_voltage_per_speed(summary_indices).', ...
    peak_current_per_speed(summary_indices).', ...
    feasible_reference(summary_indices).', ...
    VariableNames={'J_L', 'PeakVoltagePerSpeed', ...
    'PeakCurrentPerSpeed', 'MaximumReferenceStep'})
```

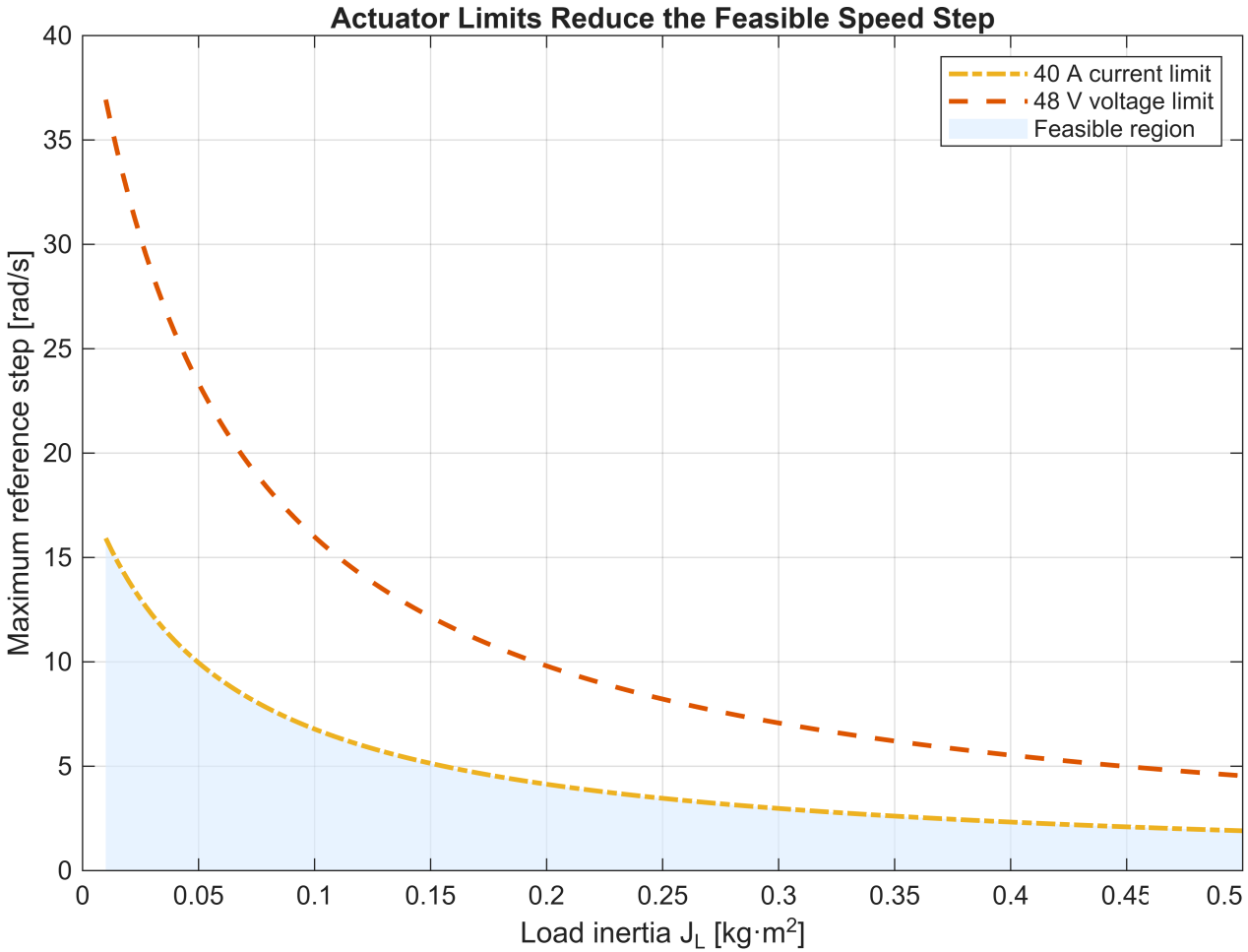
feasibility_summary = 4×4 table

	J_L	PeakVoltagePerSpeed	PeakCurrentPerSpeed	MaximumReferenceStep
1	0.0100	1.2997	2.5123	15.9214
2	0.0500	2.0565	4.0179	9.9554
3	0.2000	4.8947	9.6667	4.1379

	J_L	PeakVoltagePerSpeed	PeakCurrentPerSpeed	MaximumReferenceStep
4	0.5000	10.5709	20.9658	1.9079

Table units are kg·m², V/(rad/s), A/(rad/s), and rad/s, respectively. Replace the illustrative 40 A limit with the rating for the target inverter and motor.

```
feasibility_figure = figure(Name='Actuator Feasibility', ...
    Position=[180 180 760 520]);
area(J_L_feasibility, feasible_reference, ...
    FaceColor=[0.85 0.92 1.00], FaceAlpha=0.6, ...
    EdgeColor='none', DisplayName='Feasible region')
hold on
plot(J_L_feasibility, voltage_limited_reference, '--', ...
    LineWidth=1.8, DisplayName='48 V voltage limit')
plot(J_L_feasibility, current_limited_reference, '-.', ...
    LineWidth=1.8, DisplayName='40 A current limit')
hold off
xlabel('Load inertia J_L [kg·m^2]')
ylabel('Maximum reference step [rad/s]')
title('Actuator Limits Reduce the Feasible Speed Step')
legend(Location='northeast')
grid on
```



The feasible command lies below both curves. Beyond this envelope, voltage or current saturation changes the closed-loop dynamics, so the ideal pole locations and settling time no longer describe the physical response. Smoother reference commands can reduce these transient peaks.

5. Code Generation: Deployable Controller Functions

Generate MATLAB® and C code from the symbolic expressions for use in simulation. These functions take J_L as input and return the correct gains.

```
matlabFunction(K_of_JL, 'File', 'controllerGains', ...
    'Vars', {J_L}, ...
    'Outputs', {'K'});
matlabFunction(Nbar_of_JL, 'File', 'referenceScaling', ...
    'Vars', {J_L}, ...
    'Outputs', {'Nbar'});
fprintf('\n--- Generated controllerGains.m ---\n')
```

```
--- Generated controllerGains.m ---
```

```
type controllerGains.m
```

```
function K = controllerGains(J_L)
%controllerGains
%    K = controllerGains(J_L)

%    This function was generated by the Symbolic Math Toolbox version 26.1.
%    02-Sep-2026 11:05:20

t2 = J_L.^2;
t3 = 1.0./t2;
K = [(t3.*(J_L.*-4.24016e+6+t2.*4.78336e+7+1.0201e+4))./1.6e+7,(J_L.*4.16e+3-1.01e+2)./(J_L.*4.0e+4),t3.*(J_L.*2.2242e+6+t2.*3.36e+7-1.0201e+4).*(-6.25e
end
```

```
fprintf('\n--- Generated referenceScaling.m ---\n')
```

```
--- Generated referenceScaling.m ---
```

```
type referenceScaling.m
```

```
function Nbar = referenceScaling(J_L)
%referenceScaling
%    Nbar = referenceScaling(J_L)

%    This function was generated by the Symbolic Math Toolbox version 26.1.
%    02-Sep-2026 11:05:20

Nbar = J_L.*(4.8e+1./5.0);
end
```

```
% C code for external use
fprintf('\n--- C code for K(J_L) ---\n')
```

```
--- C code for K(J_L) ---
```

```
ccode(K_of_JL)
```

```
ans =
    '  K_of_JL[0][0] = (1.0/(J_L*J_L)*(J_L*-4.24016E+6+(J_L*J_L)*4.78336E+7+1.0201E+4))/1.6E+7;
       K_of_JL[0][1] = (J_L*4.16E+3-1.01E+2)/(J_L*4.0E+4);
       K_of_JL[0][2] = 1.0/(J_L*J_L)*(J_L*2.2242E+6+(J_L*J_L)*3.36E+7-1.0201E+4)*(-6.25E-6);
       K_of_JL[0][3] = (1.0/(J_L*J_L)*(J_L*4.268462E+8-(J_L*J_L)*5.05472E+9+(J_L*J_L*J_L)*1.536E+10-1.030301E+6))/1.6E+9;'
```

6. Simulink Integration

The generated functions can be called directly from MATLAB Function blocks in Simulink. The model architecture:

```
                        J_L_est (from estimator, lookup, or traction module)
                          |            |               |
                          v            v               v
      r -----> [Nbar(JL)] -->(+)----> u ---> [Plant(JL)] ----> y
                             ^(-)               |
                             |                  v
                             +---- [K(JL)] <--- x (state feedback)
```

```
- controllerGains(J_L) computes the 1x4 state feedback gain vector K
- referenceScaling(J_L) computes the precompensator Nbar
- J_L can come from a lookup table, an online estimator,
  or the traction control module
```

Run buildSimulinkModel.m to create a working Simulink model that demonstrates this architecture with a traction-loss scenario (J_L switches from 0.2 to 0.02 at $t = 5$ seconds).

For direct Simulink block generation: matlabFunctionBlock('model/block', K_of_JL, 'Vars', {J_L})